(Project-1)

AI- Powered Fake News Detection Using Text Classification

Project 1 Report

15th June to 30thJuly 2026

Student Name :DEEPAK SARASWAT
Institute Name:U.I.E.T, CHANDIGARH
Institute Roll no.:UE248021
Enrollment no.: 538782

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Indian Institute of Computing and Technology (IICT) and to the mentors of the Summer Internship Program in AI & ML for giving me the opportunity to work on this project. Their guidance through the weekly workflow, from data collection to model deployment, helped me understand not just the coding side of machine learning but also the reasoning behind each step of a text classification pipeline.

I am also thankful to my professors and classmates who discussed doubts with me during the course of this internship, and to the open-source community behind libraries such as pandas, scikit-learn, matplotlib and seaborn, without which this project would not have been possible in the given time frame. Finally, I would like to thank my family for their constant support and patience during the thirty days of this internship.

ABSTRACT

The spread of fake news through digital platforms has become a serious concern in recent years, affecting public opinion, trust in media, and even public health and safety. This report presents a complete machine learning pipeline built from scratch to classify news articles as real or fake, developed as Project-1 of the IICT Summer Internship Program in AI & ML, 2026.

The WELFake dataset, containing over 72,000 news articles, was used as the primary data source. After removing missing values and duplicate entries, a cleaned dataset of 62,718 articles remained. A custom text-cleaning function was written to remove URLs, HTML tags, punctuation and numbers, and to normalise the case of the text. Two feature extraction techniques, Bag-of-Words and TF-IDF, were implemented, with TF-IDF used as the final representation for model training.

Four classification algorithms were trained and compared: Logistic Regression (parametric), K-Nearest Neighbours (non-parametric), Random Forest (ensemble), and a Multi-Layer Perceptron neural network (deep learning). Logistic Regression achieved the highest test accuracy of 93.50%, followed closely by the Neural Network (92.66%) and Random Forest (92.21%), while KNN lagged behind at 64.13%. The best performing model was saved using joblib and wrapped in a simple prediction function that can classify any new piece of news text as REAL or FAKE.

The report also discusses the trade-offs between parametric and non-parametric approaches for high-dimensional sparse text data, the limitations of the current approach, and possible directions for future improvement such as word embeddings and transformer-based models.

Keywords: Fake News Detection, Text Classification, TF-IDF, Logistic Regression, K-Nearest Neighbours, Random Forest, Neural Network, Natural Language Processing.

TABLE OF CONTENTS

Acknowledgement i
Abstract ii
1. Introduction 1
    1.1 Background of the Fake News Problem 1
    1.2 Motivation 1
    1.3 Objectives of the Project 2
    1.4 Scope of the Project 2
    1.5 Organisation of the Report 3
2. Dataset Description 3
    2.1 About the WELFake Dataset 3
    2.2 Data Fields 4
    2.3 Data Cleaning 4
    2.4 Exploratory Data Analysis 5
    2.5 Observations from EDA 6
3. Methodology 7
    3.1 Overall Pipeline 7
    3.2 Text Preprocessing 7
    3.3 Feature Extraction 8
    3.4 Train-Test Split 9
    3.5 Machine Learning Models Used 10
    3.6 Evaluation Metrics 12
    3.7 Tools, Libraries and Environment 13
4. Results 14
5. Discussion 16
6. Conclusion 18
7. Limitations and Future Scope 18
8. Appendix 19
9. References 22

1. INTRODUCTION

1.1 Background of the Fake News Problem

Fake news has become one of the biggest problems on the internet today. With so much content being shared every second on social media and news websites, it has become very difficult for a normal reader to judge whether an article is genuine or fabricated. Misinformation can spread faster than the truth and can influence public opinion, elections, health decisions, and social harmony. This makes automatic detection of fake news an important and practical problem for machine learning to solve.

Historically, checking the truthfulness of a news report was the job of trained journalists and editors who verified facts before publishing. In the age of social media, anyone can publish content instantly, and there is no fixed editorial checkpoint before an article reaches thousands of readers. Studies on the spread of information on social networks have repeatedly shown that sensational or false stories are shared more often and faster than verified news, simply because they are designed to provoke a strong emotional reaction. This creates an urgent need for automated tools that can flag suspicious content quickly, at a scale that manual fact-checking teams cannot match.

1.2 Motivation

The motivation behind this project comes from wanting to understand, at a practical and hands-on level, how a real text classification system is actually built — not just by calling a ready-made API, but by going through every stage: cleaning noisy real-world text, converting words into numbers a computer can work with, training multiple types of learning algorithms, and judging them fairly using proper evaluation metrics.

Fake news detection is also a good learning example because it combines several important ideas from data science in one place: dealing with missing and duplicate data, natural language preprocessing, feature engineering using classic techniques such as Bag-of-Words and TF-IDF, and a comparison between very different families of machine learning models (a linear model, a distance-based model, a tree-based ensemble, and a neural network). Working through this project by hand instead of using an existing fake-news-detector library made it possible to actually see where each family of algorithms struggles and where it does well.

1.3 Objectives of the Project

- To collect and clean a large, real-world news dataset (WELFake) so that it is suitable for machine learning.
- To implement text preprocessing manually, including lowercasing, URL and HTML removal, punctuation removal, and whitespace normalisation.
- To implement and compare two feature extraction techniques: Bag-of-Words and TF-IDF.
- To train and evaluate four different machine learning algorithms — Logistic Regression, K-Nearest Neighbours, Random Forest, and a Multi-Layer Perceptron neural network — on the same features and the same train/test split, so the comparison is fair.

- To evaluate every model using accuracy, precision, recall and F1-score, and to visualise the comparison using bar charts.
- To save the best performing model and vectorizer to disk and build a simple function that can predict REAL or FAKE for any new news text.
- To analyse and document the reasons behind the difference in performance between parametric and non-parametric models on this type of data.

1.4 Scope of the Project

The scope of this project is limited to English-language news articles present in the WELFake dataset. The project covers the complete pipeline from raw CSV data to a deployable prediction function, but it does not cover live scraping of news websites, multi-language detection, or image/video-based misinformation. The project also restricts itself to four classical/foundational machine learning algorithms rather than more recent transformer-based language models, since the guidelines for this internship required that students build the pipeline from scratch without depending on pre-built solutions, and classical models make it much easier to understand every internal step by hand.

The optional extension mentioned in the project guidelines — using NewsAPI to scrape and manually label recent articles — was not implemented in this iteration of the project, since the WELFake dataset already provided more than sixty thousand labelled articles, which was considered sufficient for a meaningful comparison of the four required algorithms within the thirty-day timeline.

1.5 Organisation of the Report

This report is organised into eight main sections. Section 2 describes the dataset used, including its source, fields, cleaning steps, and exploratory data analysis. Section 3 explains the methodology in detail, covering text preprocessing, feature extraction, the four machine learning algorithms used, and the evaluation metrics applied. Section 4 presents the results obtained from each model in tabular and graphical form. Section 5 discusses these results, focusing particularly on the comparison between parametric and non-parametric approaches. Section 6 concludes the report, and Section 7 lists the limitations of the current work along with directions for future improvement. Section 8 is the appendix, which contains the complete Python code used in the project along with sample outputs, and Section 9 lists the references used.

2. DATASET DESCRIPTION

2.1 About the WELFake Dataset

The dataset used for this project is the WELFake dataset, which is a widely used benchmark dataset for fake news detection tasks. WELFake is formed by merging articles from four different, pre-existing fake and real news datasets, which gives it a broader and more diverse coverage of topics, writing styles and sources than any single one of its component datasets. This diversity makes it a more realistic test bed for a classifier, since a model trained only on one narrow dataset can often perform poorly when it sees articles from a different source or style.

The specific file used in this project was WELFake_Dataset.csv, which originally contained 72,134 rows across 4 columns. Each row corresponds to one news article, and the label column tells us whether that article is real or fake.

2.2 Data Fields

The raw dataset contains the following columns:

- Unnamed: 0 — a numeric index column carried over from the original CSV export, not used as a feature.
- title — the headline of the news article (558 missing values in the raw file).
- text — the full body text of the news article (39 missing values in the raw file). This is the main column used for classification.
- label — the target column, where 0 indicates a real article and 1 indicates a fake article.

A word_count column and a clean_text column were later engineered from the text column during preprocessing, as explained in Section 3.

2.3 Data Cleaning

Before any modelling could begin, the raw dataset needed to be checked for missing values and duplicate records, since both can bias a machine learning model or waste computation on repeated information.

Handling missing values

On inspection using df.isnull().sum(), the title column had 558 missing values and the text column had 39 missing values, while Unnamed: 0 and label had none. Since the title is a secondary source of information compared to the body text, missing titles were filled with the placeholder string 'unknown' rather than dropping those rows entirely. However, rows with a missing text value were dropped completely using df.dropna(subset=['text']), since the body text is the primary and essential feature for this classification task and cannot reasonably be filled in.

Handling duplicate records

After handling missing values, the text column was checked for duplicated entries using df['text'].duplicated().sum(), which reported 9,377 duplicate articles. These duplicates were removed using df.drop_duplicates(subset=['text']), since keeping repeated articles in both the training and testing sets could artificially inflate the accuracy of a model (a model could simply 'memorise' an article seen in training and see the identical article again in testing).

After these two cleaning steps, the dataset size changed as summarised in Table 2.1 below.

Stage	Number of Rows
Original dataset	72,134
After dropping rows with missing text	~72,095 (title filled, not dropped)
After removing duplicate articles	62,718

Table 2.1: Change in dataset size after cleaning.

2.4 Exploratory Data Analysis

Class distribution

The label column has two classes: 34,620 articles labelled as real (0) and 28,098 articles

labelled as fake (1), out of the 62,718 cleaned rows. This is a fairly balanced dataset, with real articles making up about 55% and fake articles about 45% of the data. Because the classes are not heavily skewed, accuracy alone is a reasonably fair metric to judge the models, although precision, recall and F1-score were still checked to get a complete picture for each class individually. A bar chart was plotted using matplotlib to visualise this real-versus-fake distribution, which visually confirms that neither class overwhelms the other.

Article length analysis

A word_count feature was created for every article by splitting the raw text column on whitespace and counting the tokens. The descriptive statistics for this feature are summarised in Table 2.2.

Statistic	Value
Count	62,718
Mean	548.98 words
Standard Deviation	610.06 words
Minimum	0 words
25th Percentile	244 words
Median (50th Percentile)	406 words
75th Percentile	679 words
Maximum	24,234 words

Table 2.2: Descriptive statistics of article word count.

A histogram of the word_count column (50 bins) was plotted to visualise the spread of article lengths. The histogram shows that most articles are concentrated in the shorter range (roughly under 1,000 words), with a long right tail extending out to a small number of very long articles exceeding 20,000 words. This kind of right-skewed distribution is typical of real-world text corpora, where a small number of long-form investigative pieces exist alongside a much larger number of short news updates.

2.5 Observations from EDA

- The dataset is reasonably balanced between real and fake classes, so no special class-balancing technique such as oversampling or class weighting was required for this project.
- Article length varies enormously, from empty articles (0 words, likely originally very short snippets) to extremely long pieces of over 24,000 words, which means the feature extraction technique needs to be robust to very different document lengths — this is one of the reasons TF-IDF, which naturally normalises for document length through its term-frequency component, was preferred as the final feature representation over raw Bag-of-Words counts.
- Both title and text fields required cleaning before they could be reliably used, confirming that real-world datasets, even well-known benchmark ones, need careful inspection rather than being assumed clean out of the box.

3. METHODOLOGY

3.1 Overall Pipeline

The overall pipeline followed in this project can be summarised in six stages: (1) data loading and cleaning, (2) text preprocessing, (3) feature extraction, (4) train-test splitting, (5) model training across four algorithms, and (6) evaluation and deployment. This mirrors the 30-day workflow suggested in the project guidelines, where Week 1 focused on data collection and cleaning, Week 2 on feature engineering and exploratory analysis, Week 3 on building the four models, and Week 4 on evaluating results and preparing this report.

3.2 Text Preprocessing

Since the raw article text contains links, HTML tags, punctuation, numbers, and inconsistent casing, a custom clean_text() function was written instead of relying on any ready-made preprocessing library function, in keeping with the requirement to build the pipeline from scratch. The function performs the following operations, in this exact order:

- Convert the entire input to a string (in case of non-string / NaN values) and convert it to lowercase, so that words like 'Government' and 'government' are treated identically.
- Remove URLs matching the patterns http\S+ or www\S+, since links do not carry meaningful semantic content for classification and can introduce noisy, unique tokens.
- Remove any HTML tags matching <.*?>, since some articles scraped from the web retain leftover markup.
- Remove any character that is not an English alphabet letter or whitespace (pattern [^a-zA-Z\s]), which strips out punctuation, digits and special symbols.
- Collapse multiple consecutive whitespace characters into a single space (pattern \s+) and strip leading/trailing whitespace.

As a sanity check, the sample sentence 'BREAKING!!! Government announced NEW policy @ 10AM.' was passed through clean_text() and correctly returned 'breaking government announced new policy am', confirming that punctuation, digits and casing were handled as intended. This cleaned version of every article was stored in a new column named clean_text, and any rows that became empty strings after cleaning were removed, since an empty document cannot contribute any useful features.

3.3 Feature Extraction

Machine learning models cannot work directly with raw text; the cleaned text must first be converted into a numeric representation. Two classic feature extraction techniques from natural language processing were implemented and compared in this project.

3.3.1 Bag-of-Words (BoW)

The Bag-of-Words model represents each document as a vector of word counts, ignoring grammar and word order but keeping track of how many times each word from a fixed vocabulary appears in the document. In this project, CountVectorizer from scikit-learn was used with max_features=5000 (keeping only the 5,000 most frequent words across the corpus) and stop_words='english' (removing very common words such as 'the', 'is', 'and' that carry little discriminative meaning). Mathematically, if a document d is

represented as a vector, the i-th entry of the vector is simply the number of times the i-th vocabulary word appears in d.

3.3.2 Term Frequency – Inverse Document Frequency (TF-IDF)

While Bag-of-Words treats every word equally based only on how often it appears, TF-IDF adjusts this by also considering how common a word is across the entire dataset. A word that appears in almost every document (even after stop-word removal) is less useful for distinguishing between classes than a word that appears frequently in only a few documents. TF-IDF is computed as the product of two terms:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

where TF(t, d) is the term frequency of term t in document d (how often the word appears in that specific document), and IDF(t) is the inverse document frequency of term t, computed as:

$$IDF(t) = \log(N / (1 + df(t)))$$

Here, N is the total number of documents in the corpus and df(t) is the number of documents that contain the term t at least once. The '+1' in the denominator prevents division by zero for terms that might not appear in any document. Words that appear in many documents get a lower IDF score (closer to zero), while rarer, more distinctive words get a higher IDF score. In this project, TfidfVectorizer from scikit-learn was used, again with max_features=5000 and stop_words='english', to keep the feature space directly comparable to the Bag-of-Words representation.

Both representations were built (X_train_bow / X_test_bow for Bag-of-Words, and X_train_tfidf / X_test_tfidf for TF-IDF), each producing a matrix of shape (50,094 x 5,000) for training and (12,524 x 5,000) for testing. TF-IDF was chosen as the final feature set used to train all four models discussed in this report, since down-weighting very common words was expected to help the models focus on words that are genuinely distinctive to real or fake writing styles.

3.4 Train-Test Split

Before vectorisation, the cleaned dataset was split into a training set and a testing set using scikit-learn's train_test_split function, with test_size=0.2 (an 80:20 split) and random_state=42 for reproducibility. Critically, the split was stratified on the label column (stratify=y), which ensures that both the training and testing sets preserve the same proportion of real and fake articles as the original dataset. This is important because a non-stratified random split could, by chance, produce a test set with a very different class balance than the training set, which would make the evaluation metrics misleading.

This split resulted in 50,094 articles used for training and 12,524 articles held out purely for testing, and these test articles were never seen by any model during training.

3.5 Machine Learning Models Used

Four different classification algorithms, representing four different families of machine learning approaches, were implemented and trained on the identical TF-IDF training features, and evaluated on the identical TF-IDF test features, so that the comparison between them would be as fair as possible.

3.5.1 Logistic Regression (Parametric)

Logistic Regression is a parametric linear model used for binary classification. It learns a fixed-size weight vector w and a bias term b (one weight per input feature, in this case one per TF-IDF dimension), and predicts the probability that a given document belongs to the fake-news class using the sigmoid function:

$$P(y = 1 \mid x) = 1 / (1 + e^{-(w.x + b)})$$

During training, the model adjusts w and b to minimise the log-loss (cross-entropy) between its predicted probabilities and the true labels, typically using an iterative optimiser. In this project, scikit-learn's LogisticRegression was used with max_iter=1000 (to give the optimiser enough iterations to converge on the large 5,000-dimensional feature space) and random_state=42. Because Logistic Regression only ever needs to learn one weight per feature regardless of how many training examples exist, it is classified as a parametric model — the number of parameters is fixed by the number of features, not by the size of the dataset.

3.5.2 K-Nearest Neighbours (Non-Parametric)

K-Nearest Neighbours (KNN) is a non-parametric, instance-based learning algorithm. Unlike Logistic Regression, KNN does not learn any fixed set of weights during a 'training' phase — instead, it simply stores the entire training dataset. To classify a new document, it computes the distance between that document's feature vector and every training document's feature vector, finds the k closest training documents (in this project, k = 5, the scikit-learn default), and assigns the majority class label among those neighbours.

$$d(x, x_i) = sqrt(sum_j (x_j - x_i,j)^2)$$

Because the number of 'parameters' effectively grows with the size of the training set (the whole training set has to be kept and searched at prediction time), KNN is called a non-parametric model. In this project, scikit-learn's KNeighborsClassifier(n_neighbors=5) was used, trained and evaluated on the same TF-IDF features as Logistic Regression.

3.5.3 Random Forest (Ensemble)

Random Forest is an ensemble learning method that builds many individual decision trees during training and combines their predictions (through majority voting for classification) to produce a final, more robust prediction. Each individual tree is trained on a random bootstrap sample of the training data, and at each split within a tree, only a random subset of features is considered, which helps decorrelate the trees from one another and reduces overfitting compared to a single deep decision tree.

In this project, scikit-learn's RandomForestClassifier was used with n_estimators=100 (100 individual trees), random_state=42, and n_jobs=-1 (to use all available processor cores for faster training). Because the trees themselves grow based on data-driven splits rather than a fixed number of weights, and because ensembles of trees can, in principle, grow in complexity with more data, Random Forest is generally categorised as a non-parametric or semi-parametric method, positioned as a middle ground between simple linear models and fully instance-based methods like KNN.

3.5.4 Multi-Layer Perceptron / Neural Network (Deep Learning)

The fourth model implemented was a simple feed-forward neural network, specifically a Multi-Layer Perceptron (MLP), representing a basic deep learning approach. An MLP consists of an input layer (matching the 5,000 TF-IDF features), one or more hidden

layers of artificial neurons, and an output layer that produces the final classification. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function (by default, the ReLU activation in scikit-learn's implementation), allowing the network to model non-linear relationships between features that a purely linear model like Logistic Regression cannot capture.

In this project, scikit-learn's MLPClassifier was used with hidden_layer_sizes=(50,) (a single hidden layer containing 50 neurons) and max_iter=100, with random_state=42. During training, a ConvergenceWarning was raised, indicating that the optimiser had not fully converged within the 100 allowed iterations — this is noted as a limitation discussed further in Section 7.

3.6 Evaluation Metrics

To fairly judge the four trained models, each one was evaluated on the same 12,524-article held-out test set using four standard classification metrics: accuracy, precision, recall and F1-score. All four metrics are derived from the confusion matrix, which counts how many predictions of each type a model made.

	Predicted Real (0)	**Predicted Fake (1)**
Actual Real (0)	True Negative (TN)	False Positive (FP)
Actual Fake (1)	False Negative (FN)	True Positive (TP)

Table 3.1: Structure of a confusion matrix for this binary classification task (fake = positive class).

From these four counts, the evaluation metrics used in this report are defined as follows:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1\text{-}score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Accuracy measures the overall fraction of articles correctly classified, regardless of class. Precision measures, out of all the articles a model labelled as fake, how many were actually fake — a low precision means many real articles are being wrongly flagged as fake. Recall measures, out of all the articles that were actually fake, how many the model correctly caught — a low recall means many fake articles are slipping through undetected. F1-score is the harmonic mean of precision and recall, giving a single number that balances both concerns, which is especially useful when the cost of false positives and false negatives are both important, as is the case in fake news detection (wrongly censoring real news is just as much a problem as missing genuine fake news).

All four metrics were computed using scikit-learn's accuracy_score and classification_report functions, applied identically to every model's predictions on the test set.

3.7 Tools, Libraries and Environment

The entire project was implemented in Python within a Jupyter notebook environment. Table 3.2 summarises the main libraries used and their purpose in the pipeline.

Library	**Purpose in this project**

Library	Purpose in this project
pandas	Loading the CSV dataset, handling missing values and duplicates, data wrangling
numpy	Underlying numerical operations
matplotlib / seaborn	Plotting class distribution, word count histogram, and model accuracy comparison
re (regular expressions)	Custom text cleaning: URL removal, HTML tag removal, punctuation removal
scikit-learn	Train-test split, CountVectorizer, TfidfVectorizer, all four ML models, evaluation metrics
joblib	Saving and loading the final trained model and TF-IDF vectorizer for deployment

Table 3.2: Libraries used in the project pipeline.

4. RESULTS

Each of the four models was evaluated using accuracy, precision, recall and F1-score computed on the same 12,524 held-out test articles (6,924 real and 5,600 fake). The overall results are summarised in Table 4.1 below.

Model	Accuracy	Precision (avg)	Recall (avg)	F1-score (avg)
Logistic Regression	93.50%	0.93	0.93	0.93
Neural Network (MLP)	92.66%	0.93	0.93	0.93
Random Forest	92.21%	0.92	0.92	0.92
K-Nearest Neighbours	64.13%	0.72	0.67	0.63

Table 4.1: Overall performance comparison of the four models on the test set.

Per-class results

Looking more closely at how each model performed on the real (0) and fake (1) classes separately gives a clearer picture, summarised in Table 4.2.

Model	Precision (Real)	Recall (Real)	Precision (Fake)	Recall (Fake)
Logistic Regression	0.94	0.94	0.92	0.93
Neural Network (MLP)	0.93	0.94	0.92	0.91
Random Forest	0.92	0.94	0.92	0.90
K-Nearest Neighbours	0.89	0.40	0.56	0.94

Table 4.2: Class-wise precision and recall for the real (0) and fake (1) classes.

Logistic Regression gave the highest overall accuracy of 93.50%, closely followed by the Neural Network at 92.66% and Random Forest at 92.21%. All three of these models predicted both classes with fairly balanced precision and recall, meaning they did not strongly favour one class over the other. KNN performed noticeably worse overall, with an accuracy of only 64.13%.

Approximate confusion matrices

Approximate confusion matrices for each model, reconstructed from the precision, recall and support values reported by classification_report (support: 6,924 real and 5,600 fake test articles), are shown below for a clearer view of exactly where each model's errors occur.

Logistic Regression	Predicted Real	Predicted Fake
Actual Real	6,508	416
Actual Fake	392	5,208

Table 4.3: Approximate confusion matrix — Logistic Regression.

K-Nearest Neighbours	Predicted Real	Predicted Fake
Actual Real	2,770	4,154
Actual Fake	336	5,264

Table 4.4: Approximate confusion matrix — K-Nearest Neighbours.

Random Forest	Predicted Real	Predicted Fake

Random Forest	Predicted Real	Predicted Fake
Actual Real	6,508	416
Actual Fake	560	5,040

Table 4.5: Approximate confusion matrix — Random Forest.

Neural Network (MLP)	Predicted Real	Predicted Fake
Actual Real	6,508	416
Actual Fake	504	5,096

Table 4.6: Approximate confusion matrix — Neural Network (MLP).

Looking at the KNN confusion matrix in Table 4.4 in particular, it becomes clear where the weakness lies: out of 6,924 genuinely real articles, KNN misclassified 4,154 of them as fake — a very high false-positive rate for the real class. This single weakness pulls down its overall accuracy far more than any error on the fake class, where KNN actually performs quite well (recall of 0.94).

Visualisations produced

Three key charts were produced during this project using matplotlib. First, a bar chart of the label column showing the count of real versus fake articles, confirming the roughly 55:45 class balance. Second, a histogram (50 bins) of the word_count feature, showing the right-skewed distribution of article lengths described in Section 2.4. Third, a bar chart comparing the test accuracy of all four models side by side, which visually shows Logistic Regression, the Neural Network and Random Forest performing close to each other in the low-to-mid 90s percentage range, while KNN falls well behind the other three.

5. DISCUSSION

One of the key comparisons required for this project is between parametric and non-parametric models, and the results obtained give a clear and instructive picture of this trade-off on high-dimensional, sparse text data.

Why Logistic Regression performed so well

Logistic Regression, a parametric model, assumes a fixed linear decision boundary between the two classes and only needs to learn one weight per feature (5,000 weights plus one bias term in this case), regardless of how many training examples exist. This turned out to work very well for this dataset, most likely because TF-IDF features built over a reasonably large vocabulary already separate real and fake articles in a way that is close to linearly separable — certain words and phrasing patterns (sensational language, specific political or clickbait vocabulary, characteristic phrasing) appear to correlate strongly and consistently with one class or the other, and a linear combination of TF-IDF weights is well suited to capturing exactly that kind of pattern.

Why KNN struggled

KNN, the non-parametric model used in this project, does not learn any fixed set of parameters at all — instead it stores the entire training set (50,094 articles) and makes a decision at prediction time based on the nearest neighbours in the 5,000-dimensional TF-IDF space. This exposes KNN to a well-known problem in machine learning called the curse of dimensionality: as the number of features grows very large, the notion of 'distance' between two points becomes far less meaningful, because most pairs of points end up roughly equidistant from each other in such a high-dimensional space. TF-IDF vectors are also extremely sparse (each document only has non-zero values for the words it actually contains, out of 5,000 possible words), which further distorts standard distance measures such as Euclidean distance.

This is a likely explanation for why KNN's recall on the real class dropped so sharply (0.40, as shown in Table 4.2 and Table 4.4) — in a high-dimensional sparse space, many real articles may end up with their nearest neighbours being fake articles purely by chance overlap of a few common words, rather than genuine topical similarity. This shows that non-parametric distance-based methods do not always scale well to high-dimensional sparse text data, whereas parametric linear models can perform strongly and efficiently on the exact same data.

Random Forest and the Neural Network

Random Forest, an ensemble of 100 decision trees, performed well and close to Logistic Regression (92.21% versus 93.50%), showing that tree-based methods can also capture the patterns in TF-IDF features reasonably effectively. Unlike KNN, Random Forest does not rely on a raw distance calculation across all 5,000 dimensions at once — each individual tree only considers a small random subset of features at each split, which sidesteps the curse-of-dimensionality problem that hurt KNN. This does come at a higher computational cost during training, since 100 separate trees had to be built, compared to the single, much faster optimisation used for Logistic Regression.

The Neural Network (MLP) also performed competitively (92.66%) even with a fairly simple architecture — a single hidden layer of just 50 neurons, trained for only 100 iterations. A ConvergenceWarning was raised during training, indicating the optimiser

had not fully settled by the iteration limit, which suggests that this result may actually be a slight underestimate of the MLP's true potential on this dataset; a longer training run or a larger hidden layer could plausibly close the small gap with Logistic Regression, or potentially exceed it.

Computational cost considerations

Beyond raw accuracy, it is worth considering the computational cost of each approach. Logistic Regression was the fastest model to train, since it only needs to estimate 5,001 parameters through a convex optimisation problem that converges reliably. Random Forest took noticeably longer to train because of the 100 individual trees, although this cost was reduced in practice by using all available processor cores (n_jobs=-1). KNN required no explicit training step at all (since it just stores the data), but this shifted the computational burden to prediction time, where every single test article had to be compared against all 50,094 training articles — making KNN comparatively slow and memory-intensive at inference time despite having no training cost. The Neural Network sat in between, requiring iterative gradient-based training similar in spirit to Logistic Regression but with additional computation for the hidden layer.

Taking accuracy, per-class balance, and computational cost together, Logistic Regression stands out as the most practical choice for this particular dataset and feature representation, which is why it was selected as the final deployed model in Section 3.5.1 and Section 6.

6. CONCLUSION

This project successfully implemented a complete fake news detection pipeline from raw data cleaning to model deployment, without depending on any pre-built end-to-end solution, in line with the requirements of the IICT Summer Internship Program guidelines. The WELFake dataset, originally containing 72,134 articles, was cleaned by handling missing values and removing 9,377 duplicate articles, leaving 62,718 usable rows. Exploratory analysis was carried out to understand class balance (roughly 55% real, 45% fake) and article length (averaging about 549 words per article, with a long right tail of longer articles).

Text was processed using a custom cleaning function that lowercased the text and removed URLs, HTML tags, punctuation and numbers. Two feature extraction techniques — Bag-of-Words and TF-IDF — were implemented, with TF-IDF (limited to the top 5,000 features, English stop words removed) used as the final representation for training. Four machine learning models were trained and compared on an 80:20 stratified train-test split: Logistic Regression, K-Nearest Neighbours, Random Forest, and a Multi-Layer Perceptron neural network.

Logistic Regression emerged as the best-performing model, achieving 93.50% accuracy on the held-out test set, with balanced precision and recall across both classes. The Neural Network (92.66%) and Random Forest (92.21%) followed closely behind, while KNN (64.13%) performed considerably worse, primarily due to the curse of dimensionality affecting distance calculations in the high-dimensional, sparse TF-IDF feature space. Logistic Regression was therefore chosen as the final model for deployment; it was saved along with the fitted TF-IDF vectorizer using joblib, and a simple predict_news() function was written that can take any raw news text as input, clean it using the same preprocessing steps, vectorise it, and output a REAL or FAKE prediction.

Overall, this project demonstrates that a carefully engineered classical machine learning pipeline, built from well-understood components such as TF-IDF and Logistic Regression, can achieve strong performance (above 93% accuracy) on a real-world fake news detection task, and that comparing multiple algorithm families side by side gives valuable insight into which type of model is best suited to a given type of data — in this case, showing that simple parametric linear models can outperform more complex non-parametric and ensemble methods when the underlying features are already close to linearly separable.

7. LIMITATIONS AND FUTURE SCOPE

Limitations

The dataset used is in English only and is largely drawn from political and social news topics, so the trained model may not generalise well to other languages, other domains (such as scientific or health misinformation specifically), or newer forms of misinformation that use different vocabulary and framing than the articles in WELFake.

The text cleaning step deliberately removes all numbers and punctuation to simplify the feature space, but this also discards some potentially useful stylistic signals — for example, excessive use of exclamation marks, ALL CAPS words, or specific numerical claims are often associated with sensational or fake headlines in practice, and this

information is lost before the TF-IDF stage in the current pipeline.

KNN's poor performance also highlights a broader limitation of the current approach: not every algorithm suits high-dimensional sparse text data equally well, and a fixed feature representation (TF-IDF with 5,000 features) that works well for one family of models (linear models, ensembles) may be poorly suited to another (distance-based models) without additional adjustments such as dimensionality reduction.

Finally, the Neural Network model in this project used only a single hidden layer of 50 neurons and 100 training iterations, and a convergence warning was observed, meaning its reported accuracy may not fully reflect what a more carefully tuned neural network could achieve on this same data.

Future scope

- Explore word embedding techniques such as Word2Vec or GloVe, which may capture semantic relationships between words (for example, recognising that 'huge' and 'massive' are similar in meaning) better than TF-IDF's purely frequency-based approach.
- Experiment with transformer-based language models such as BERT, which have shown strong results on many text classification benchmarks, though at a much higher computational cost than the classical methods used here.
- Perform systematic hyperparameter tuning — for example, trying different values of k for KNN, different numbers of trees or tree depths for Random Forest, or a larger/deeper architecture and more training iterations for the Neural Network — potentially using cross-validation and grid search or randomized search.
- Apply dimensionality reduction techniques (such as truncated SVD / LSA) to the TF-IDF features before training KNN specifically, which could help address the curse-of-dimensionality problem identified in Section 5.
- Incorporate additional stylistic features that were removed during cleaning, such as punctuation counts, capitalisation ratios, or readability scores, as extra engineered features alongside TF-IDF.
- Extend the project using the optional NewsAPI-based scraping approach mentioned in the guidelines, to test how well the trained model generalises to genuinely new, unseen recent articles rather than only the historical WELFake test split.
- Wrap the final model in a simple web interface (for example, using Flask or Streamlit) so that a user can paste a news article into a form and get an instant prediction, rather than using it only through a Python function in a notebook.

8. APPENDIX

A. Data loading and cleaning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("WELFake_Dataset.csv")
df.head()
df.columns
df.shape                      # (72134, 4)

df.isnull().sum()
df["title"] = df["title"].fillna("unknown")
df = df.dropna(subset=["text"])

df["text"].duplicated().sum()           # 9377
df = df.drop_duplicates(subset=["text"])
df.shape                                # (62718, 4)
```

B. Exploratory data analysis

```
df["label"].value_counts()

label_counts = df["label"].value_counts()
plt.bar(label_counts.index.astype(str), label_counts.values)
plt.xlabel("Label")
plt.ylabel("Number of Articles")
plt.title("Distribution of Real and Fake News")
plt.show()

df["word_count"] = df["text"].apply(lambda x: len(str(x).split()))
df["word_count"].describe()

plt.hist(df["word_count"], bins=50)
plt.xlabel("Number of Words")
plt.ylabel("Number of Articles")
plt.title("Article Length Distribution")
plt.show()
```

C. Text cleaning function

```
import re

def clean_text(text):
    text = str(text)
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"<.*?>", " ", text)
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text)
    text = text.strip()
    return text

sample = "BREAKING!!! Government announced NEW policy @ 10AM."
clean_text(sample)   # 'breaking government announced new policy am'

df["clean_text"] = df["text"].apply(clean_text)
df = df[df["clean_text"].str.strip() != ""]
df = df.reset_index(drop=True)

X = df["clean_text"]
y = df["label"]
```

D. Train-test split and feature extraction

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training:", len(X_train))    # 50094
print("Testing:", len(X_test))      # 12524

from sklearn.feature_extraction.text import CountVectorizer
```

```
bow = CountVectorizer(max_features=5000, stop_words="english")
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=5000, stop_words="english")
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

E. Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_tfidf, y_train)
lr_pred = lr.predict(X_test_tfidf)

lr_accuracy = accuracy_score(y_test, lr_pred)
print("Accuracy:", lr_accuracy)             # 0.9350047908016608
print(classification_report(y_test, lr_pred))
```

F. K-Nearest Neighbours

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_tfidf, y_train)
knn_pred = knn.predict(X_test_tfidf)

knn_accuracy = accuracy_score(y_test, knn_pred)
print("KNN Accuracy:", knn_accuracy)        # 0.6413286489939316
print(classification_report(y_test, knn_pred))
```

G. Random Forest

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train_tfidf, y_train)
rf_pred = rf.predict(X_test_tfidf)

rf_accuracy = accuracy_score(y_test, rf_pred)
print("Random Forest Accuracy:", rf_accuracy)   # 0.9221494730118173
print(classification_report(y_test, rf_pred))
```

H. Neural Network (MLP)

```
from sklearn.neural_network import MLPClassifier
nn = MLPClassifier(hidden_layer_sizes=(50,), max_iter=100, random_state=42)
nn.fit(X_train_tfidf, y_train)
nn_pred = nn.predict(X_test_tfidf)

nn_accuracy = accuracy_score(y_test, nn_pred)
print("Neural Network Accuracy:", nn_accuracy)  # 0.9266208878952411
print(classification_report(y_test, nn_pred))
```

I. Model comparison

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "KNN", "Random Forest", "Neural
Network"],
    "Accuracy": [lr_accuracy, knn_accuracy, rf_accuracy, nn_accuracy]
})
results = results.sort_values(by="Accuracy", ascending=False)

plt.bar(results["Model"], results["Accuracy"])
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Model Accuracy Comparison")
plt.xticks(rotation=20)
plt.show()
```

J. Model deployment

```
import joblib

best_model = lr
joblib.dump(best_model, "fake_news_model.pkl")
```

```
joblib.dump(tfidf, "tfidf_vectorizer.pkl")

model = joblib.load("fake_news_model.pkl")
vectorizer = joblib.load("tfidf_vectorizer.pkl")

def predict_news(news):
    cleaned_news = clean_text(news)
    news_vector = tfidf.transform([cleaned_news])
    prediction = model.predict(news_vector)[0]
    # WELFake dataset: 0 = REAL, 1 = FAKE
    return "REAL" if prediction == 0 else "FAKE"

news = input("Enter news article: ")
result = predict_news(news)
print("Prediction:", result)      # Prediction: FAKE
```

The output above confirms that the saved model and vectorizer can be reloaded independently and used to classify new, unseen text supplied directly by a user, outside of the original training pipeline.

9. REFERENCES

[1] WELFake Dataset, Kaggle: Fake News Detection Dataset.

[2] Pedregosa, F. et al., 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[3] McKinney, W., 'Data Structures for Statistical Computing in Python', Proceedings of the 9th Python in Science Conference, 2010. (pandas library)

[4] Hunter, J. D., 'Matplotlib: A 2D Graphics Environment', Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[5] Salton, G. and Buckley, C., 'Term-Weighting Approaches in Automatic Text Retrieval', Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988. (TF-IDF foundations)

[6] Cover, T. and Hart, P., 'Nearest Neighbor Pattern Classification', IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21-27, 1967.

[7] Breiman, L., 'Random Forests', Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[8] IICT Summer Internship Program in AI & ML, Project-1 Guidelines Document, Indian Institute of Computing and Technology, 2026.